How-To Library

Informatica

PowerExchange Code Page Processing

© Copyright Informatica LLC 2016, 2021. Informatica LLC. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC. All other company and product names may be trade names or trademarks of their respective owners and/or copyrighted materials of such owners.

Abstract

PowerExchange supports code pages for internationalization. This article discusses various aspects of PowerExchange code page processing.

Supported Versions

- PowerExchange 9.6.1, 10.0, 10.1

Table of Contents

Code Pages and PowerExchange Client-Server Architecture. . . . 3
PowerExchange Architecture Overview. . . . 3
Code Page Values for the PowerExchange Listener. . . . 4
Code Page Values for Client Applications. . . . 5
Metadata Code Page. . . . 5
PowerExchange Internal Code Page Numbers. . . . 6
Finding an Internal Code Page Number from a Name. . . . 6
Code Pages Used by Numeric Column Types. . . . 7
Code Page Conversions During PowerCenter Workflow Processing. . . . 8
Workflow Processing Overview. . . . 8
Step 1. Issue and Process an Open Request. . . . 8
Step 2. Describe Columns. . . . 8
Step 3. Determine the Client Data Code Page. . . . 9
Step 4. Bind Column Buffers. . . . 9
Step 5. Set Up PowerExchange API Conversions. . . . 10
Step 6. Perform PowerExchange Code Page Conversions. . . . 10
Step 7. Perform PowerCenter Code Page Conversions. . . . 10
Step 8. Perform RDBMS Code Page Conversions. . . . 11
Relational Access Methods That Describe Columns. . . . 11
Describing Columns in DB2 for Linux, UNIX, and Windows. . . . 11
Describing Columns in Microsoft SQL Server. . . . 11
Describing Columns in Oracle. . . . 11
Describing Columns in DB2 for z/OS. . . . 12
Nonrelational Access Methods. . . . 14
NRDB Description of Character Columns from Record Fields. . . . 14
NRDB Description of Character Columns from User-Defined Fields. . . . 14
Special NRDB Situations. . . . 14
z/OS Considerations. . . . 15
DB2 for z/OS ECCR. . . . 15
Single-Byte Metadata Limitation. . . . 15
PMICU Usage on z/OS. . . . 16

PMICU. . . . 16
PMICU Background. . . . 16
Substitution Characters. . . . 16
Supplemental Characters. . . . 17
Customized ICU Code Pages. . . . 18
Non-ICU Code Pages. . . . 18
Code Page Usage by Country, Language, and Type. . . . 19
Code Page Usage Reports. . . . 19
EBCDIC Code Pages that Support the Euro Sign. . . . 19
Common Single-Byte Code Pages. . . . 20
Turkish EBCDIC Code Pages. . . . 20
Japanese EBCDIC Code Pages. . . . 20
Arabic and Hebrew EBCDIC. . . . 21
Issues That Have Workarounds. . . . 22
Non-conversion of Control Characters. . . . 22
Truncation of Strings at the First Binary Zero Character. . . . 22
Unable to Start ASCII Mode Integration Service in Certain Code Pages. . . . 23
Limitations. . . . 23
Unable to Truncate Multibyte Column Data. . . . 23
Multibyte Precision Not Known After Conversion. . . . 23
Unable to Process Different Code Pages Inside a Single Column. . . . 24
Frequently Asked Questions. . . . 24
Where are code page conversions performed?. . . . 24
What is the recommended data movement mode for the Integration Service?. . . . 25
Can PowerExchange read multibyte file names?. . . . 25
Can the PowerExchange Navigator display text in a language for which PowerCenter is not localized?. . . . 26
Can PowerExchange process multibyte Asian data on a U.S. localized machine?. . . . 26
What are the Unicode code pages to use and to avoid?. . . . 26
How many bytes does a wchar_t character contain?. . . . 27
Appendix A: EBCDIC Metadata Characters outside US_ASCII. . . . 27

Code Pages and PowerExchange Client-Server Architecture

PowerExchange Architecture Overview

Many client PowerExchange applications can communicate through sockets across a network to access methods running under a PowerExchange Listener.

Example client applications include:

- PowerExchange Navigator
- PowerExchange utilitites, such as DTLUAPPL, DTLUCBRG, DTLURDMO, and PWXUCDCT

- PowerCenter PWXPC connections to the Listener through the PowerExchange Call Level Interface, DTLSCLI
- PowerCenter ODBC connections to the Listener through the PowerExchange ODBC Interface, DTLODBC

Generally, the code page of character data is defined by the following control fields:

Control Field	Usage
Control code page	Code page of internal control blocks, such as: - Names of databases, tables, and files - Substitution values in messages
Data code page	Default code page for column data if not overridden
SQL code page	Code page of SQL

Code Page Values for the PowerExchange Listener

The PowerExchange Listener gets the values for the control, data, and SQL code pages from the CODEPAGE statement in the DBMOVER configuration file.

When a Listener subtask starts, it informs the client session of its control and SQL code page values. The client session performs code page conversion of user ID, password, database name, and table name values and of SQL statements. The client session then sends the Open request in the format in which the Listener subtask expects it.

The control and SQL code pages must be able to hold the characters of names or SQL that are being processed. If a single-byte code page is used and an attempt is made to process a multibyte name, message PWX-01291 is logged, and the process is aborted.

Specify the CODEPAGE statement for the Listener under either of the following conditions:

- Accented characters or other single-byte characters outside of the 7-bit range are used, such as pound signs or yen signs.
- Multibyte characters are used. Note that the control and SQL code pages on EBCDIC machines can be single byte only.

When setting these code page values, use the following guidelines:

Control code page

It is important to choose the appropriate control code page. PowerExchange Open requests are aborted if any substitution of replacement characters occurs.

On Linux, UNIX, and Windows, UTF-8 is a good choice because it supports the entire basic plain Unicode range, and it matches the code page in which data maps and other PowerExchange metadata are stored.

On EBCDIC platforms, IBM-037 is a good choice because it matches the code page in which data maps and other PowerExchange metadata are stored. However, in certain situations, support might be required for country-specific characters in files names.

Data code page

The data code page is less important than the control and SQL code pages. Typically, you can set it to the same value as the control and SQL code pages.

SQL code page

The SQL code page sometimes needs to be set according to the requirements of the database system. For example, you might need to set it to match one of the following values:

- Code page used by a DB2 for z/OS subsystem
- DB2CODEPAGE environment variable for DB2 for Linux, UNIX, and Windows
- NLS_LANG environment variable for Oracle

If all of the data lies in the 7-bit ASCII range, the default code pages work adequately. If you do not specify the CODEPAGE statement, the default is ISO-8859 for Linux, UNIX, and Windows and IBM-037 for EBCDIC platforms.

Code Page Values for Client Applications

In PowerExchange releases earlier than 8.5.1, PowerExchange client applications used the control, data, and SQL code page values from the CODEPAGE statement in the DBMOVER configuration file in the same way that the PowerExchange Llistener does. However, across several releases beginning with PowerExchange 8.5.1, PowerExchange client applications were changed to override the values in the CODEPAGE statement.

PowerExchange 9.6.1 and later client applications use the following code pages:

Client	Code Page Override
PowerExchange Navigator	Control, data, and SQL code pages are set to UTF-8.
DTLUCBRG utility DTLURDMO utility	Control, data, and SQL code pages are set to match the metadata code page.
PWXPC connection through DTLSCLI	Control and SQL code pages are set to UTF-8. In ASCII mode, the data control page is set to the PowerCenter Integration Service code page. In Unicode mode, the data code page is set to the connection code page.
ODBC connection through DTLODBC	The data code page is set to match the ODBC.INI LOCALCODEPAGE number. **Note:** Verifying that the ODBC.INI code page matches the one used by PowerCenter is a manual process. Neither DTLODBC nor PMODBC can check that the configuration is correct.

Metadata Code Page

Data in data maps, CDC extraction maps, and PowerExchange control files (such as CCT, CDCT, and CDEP) is stored in the metadata code page for the machine type.

Machine Type	Metadata Code Page
i5/OS	IBM-037
Linux, UNIX, and Windows	UTF-8
z/OS	IBM-1047

The following code pages contain the same characters: IBM-037, IBM-1047, and ISO-8859.

In the IBM-037 and IBM-1047 code pages, the left and right square bracket characters ([ ]) are located at different hexadecimal values. PowerExchange makes use of these characters in data maps when defining arrays.

Because the ASCII ISO-8859 code page contains the same 256 characters as IBM-037, you can use ISO-8859 for the for PowerCenter Integration Service or connection code pages instead of MS1252.

PowerExchange currently has no facility to store multibyte metadata on i5/OS or z/OS.

PowerExchange Internal Code Page Numbers

PowerExchange uses numbers to uniquely define code pages. The PowerExchange code defines an array of 340 entries.

The array is subdivided into the following ranges:

- 1 to 40 for the static single-byte code pages that are shipped with PowerExchange.
- 41 to 299 for the PMICU code pages that are shipped with PowerExchange. The converters are contained in the PMICU data library.
- 301 to 339 for optional PMICU customized code pages. The run-time converters are built as .CNV files from source .UCM files that map characters to and from Unicode.

The ICUCHECK utility reports the code page control array and lists information for each internal code page number, including:

- ICU converter name, such as IBM-5348_P100-1997
- PM locale short name, such as MS1252
- PM long description, such as MS Windows Latin 1 (ANSI), superset of Latin1

Existing PowerExchange internal code page numbers never change, but sometimes extra code pages are added.

You can change the code page definitions by using the ICUALIAS, ICUCONVERTER, and ICUCNVPROPERTY statements in the DBMOVER configuration file. These statements are necessary when you use a customized ICU code page. In PowerExchange releases earlier than 9.6.0, these statements were sometimes used to handle problem DB2 CCSID mappings. However, the DB2CODEPAGE statement, introduced in PowerExchange 9.6.0, is now recommended instead.

Finding an Internal Code Page Number from a Name

In various contexts, a PowerExchange code page lookup routine is passed a name and returns an internal code page number.

The lookup routine tolerates near matches by using the following techniques:

- Punctuation characters such as hyphen (-) and underscore (_) are removed from the names during matching. For example, UTF-8 matches with UTF8.
- Case is ignored. For example, UTF-8 matches with utf-8.
- The incoming name is matched against names for ICU converters, PM locale short names, PM long descriptions, and aliases.

For example, the lookup routine would return CPN 153 from the following names:

- IBM-5348_P100-1997 (ICU converter name)
- MS1252 (PM locale short name)
- MS Windows Latin 1 (ANSI), superset of Latin1 (PM long description)
- CP1252 (DB2 CCSID)

Code Pages Used by Numeric Column Types

The following PowerExchange column types are classified as numeric and take the code page in which PowerExchange programs are compiled:

- ZONED, UZONED
- NUMCHAR
- DATE
- TIME, TIMESTAMP, EXTMSTAMP, TIMESTMEP

PowerExchange programs are compiled in the following code pages:

- ISO-8859 on Linux, UNIX and Windows
- IBM-037 on i5/OS
- IBM-1047 on z/OS

Notes:

- ISO-8859, IBM-037 and IBM-1047 all contain the same characters when expressed as Unicode or character names. Only the hexadecimal values are different.
- Numeric fields use a subset of characters (including 0-9, comma, period, plus, and minus). These characters are the same in IBM-037 and IBM-1047.

When the PowerExchange API reads numeric values, before performing datatype conversions, it converts numeric column data to the code page in which programs are compiled on the machine.

When the PowerExchange API writes numeric values, it works in the code page in which programs are compiled on the machine. As the last step, PowerExchange converts the code page to the code page that is required on the remote Listener or access method.

Packed decimal types do not need any code page conversion, as the hexadecimal representation is the same on all machines.

The sign byte on ZONED and UZONED numbers can take hexadecimal values of C*y*, D*y*, and F*y*, where *y* represents a 4-bit hexadecimal value.

Zoned numbers that end in a plus sign, minus sign, or unsigned zero are treated as follows:

- X'C0' means plus zero.
  - In EBCDIC, this value represents <U007B> { LEFT CURLY BRACKET.
  - In ISO-8859, this value represents <U00C0> À LATIN CAPITAL LETTER A WITH GRAVE.
- X'D0' means minus zero.
  - In EBCDIC, this value represents <U007D> } RIGHT CURLY BRACKET.
  - In ISO-8859, this value represents <U00D0> Ð LATIN CAPITAL LETTER ETH.
- X'F0' means unsigned zero.
  - In EBCDIC, this value represents <U0030> 0 DIGIT ZERO.

- In ISO-8859, this value represents <U00F0> ð LATIN SMALL LETTER ETH.

Code Page Conversions During PowerCenter Workflow Processing

Workflow Processing Overview

When a PowerCenter workflow runs, the following code page conversion processing occurs:

1. Issue and process an Open request.
2. Describe columns.
3. Determine the client data code page.
4. Bind column buffers.
5. Set up PowerExchange API conversions.
6. Perform PowerExchange code page conversions.
7. Perform PowerCenter code page conversions.
8. Perform RDBMS code page conversions.

Step 1. Issue and Process an Open Request

When an Open request is issued, a handshake between the client and PowerExchange Listener machines is performed.

1. The client machine makes a sockets connection to the Listener.
2. The Listener passes its control, data, and SQL code pages to the client.
3. The client converts user credentials and database information to the Listener control code page. If a code page conversion fails, processing is aborted.
4. The Listener performs Open processing and prepares for the Describe call.

Step 2. Describe Columns

On the Listener machine, the PowerExchange access method for the source describes the columns and sets the following attributes:

- Column type and length
- PowerExchange internal code page number for character data with the CHAR and VARCHAR datatypes

The client issues a Describe call to the PowerExchange access method to get these attributes in the following situations:

- When importing metadata, such as when you select the **Import from PowerExchange** command in the PowerCenter Designer
- When you initialize a workflow

This process assumes that the NLS_LANG and DB2CODEPAGE environment variables are the same on the machine that is used for importing metadata as on the Integration Service machine.

Step 3. Determine the Client Data Code Page

The second part of Describe processing determines the code page in which PowerExchange exchanges column data with PowerCenter on the Integration Service machine.

In a PWXPC workflow, the code page in which PowerExchange exchanges character data is derived from one of the following sources:

- Code page of the Integration Service if the Integration Service is running in ASCII mode
- Code page of the connection if the Integration Service is running in Unicode mode

In an ODBC workflow, the code page is derived from the LOCALCODEPAGE value in the ODBC.INI file.

ODBC Workflows

For ODBC workflows, the same code page must be specified in the following locations:

- In Workflow Manager, the connection Code Page field, which is used by the PowerCenter PMODBC program.
- In the ODBC.INI file, the LOCALCODEPAGE parameter, which is used by PowerExchangae to determine the client data code page.

When you configure ODBC on Windows, the DTLODBCW GUI automaticallly inserts the required code page number in the LOCALCODEPAGE parameter.

When you configure ODBC on Linux or UNIX, more manual effort is required. The simplest method is to configure ODBC on Windows, export Windows ODBC definitions to a flat file, and copy this file to the Linux or UNIX machine. Otherwise, you must use the ICUCHECK report to determine the required internal code page number and insert this value into the LOCALCODEPAGE parameter.

Messages PWX-07122 and PWX-07130

PowerExchange reports the internal code page numbers in the PWX-07122 and PWX-07130 messages. If both messages are logged, the value in the latter message takes precedence.

When the PowerExchange Call Level Interface (SCLI) receives the connect string, it logs the current data code page in the PWX-07122 message. This code page is typically the same as the data code page in the CODEPAGE statement in the DBMOVER configuration file and might be superseded later.

Example PWX-07122 message:

```
PWX-07122 DTLSCLI connected using DTLConnect PWX Version: 9.5.1, Patch Level: DEV_BUILD, Local
code pages: Control=UTF-8 (41) Data=UTF-8 (41) SQL=UTF-8 (41).
```

The code page that a PWXPC workflow uses is then set. If the work code page is different from the code page in message PWX-07122, message PWX-07130 is issued.

Example PWX-7130 message:

```
PWX-07130 Data Code Page reset to UTF-16 encoding of Unicode (Lower Endian) (44).
```

Step 4. Bind Column Buffers

The PowerExchange column types are derived from how the calling application binds the result set columns according to the binding APIs used by PWXPC or ODBC.

Binding might be performed during initialization or might be deferred until the first read or write.

If multibyte code pages are involved, the system validates that the caller has provided buffers that are large enough to hold data after expansion. If the column size in the source object in the mapping is too small, the workflow might be aborted with error message PWX-07096.

Example PWX-07096 message:

```
07096 column number (name) caller buffer length buffer_length less than the minimum minimum for
multibyte code page pwx_code_page_number (name).
```

If the system determines that the caller has not provided a big enough buffer, perform one of the following actions:

- Run the workflow in Unicode mode.
- Expand the length of the problem column in the source or target object to the minimum value from the PWX-07096 message.
- Use a different code page to avoid possible expansion during code page conversion.

For more information about PWX-07096, see "Limitations" on page 23.

Step 5. Set Up PowerExchange API Conversions

On the first read or write call, PowerExchange defines conversions for each column with respect to column datatypes, lengths, and code pages. Thereafter, the conversion style remains the same for all subsequent rows.

For CHAR and VARCHAR columns, this conversion involves initializing translator objects between two code pages. The conversion occurs from the access method code page to the client code page when reading data and in the opposite direction when writing data.

When numeric columns types such as PACKED or ZONED are converted to strings for the caller (which occurs when you enable high precision in a PowerCenter session), the numeric outputs are passed to the caller in the code page in which the programs are compiled (ISO-8859). In this situation, the code page of the caller (for example, UTF-16BE, UTF-16LE, or an EBCDIC code page) is ignored.

Step 6. Perform PowerExchange Code Page Conversions

PowerExchange performs code page conversions on the Integration Service machine when reading or writing multibyte data.

When reading multibyte data, PowerExchange performs the following code page conversions:

- From the source code page to intermediate Unicode
- From intermediate Unicode to the code page of the caller, which can be the Integration Service, the connection, or ODBC.INI.

When writing multibyte data, PowerExchange performs the following code page conversions:

- From the code page of the caller to intermediate Unicode
- From intermediate Unicode to the target code page

The processing includes the following optimizations:

- When the caller uses a single-byte code page, all 256 source bytes are converted using intermediate Unicode to build a translate array, and then the translate array is used for subsequent conversions.
- When the caller uses the native Unicode for the machine (UTF-16BE or UTF-16LE), either the ucnv_toUChars() or ucnv_fromUChars() function is performed, but not both.

Step 7. Perform PowerCenter Code Page Conversions

When the Integration Service runs in Unicode mode, an additional code page conversion is performed from the connection code page back to UTF-16BE or UTF-16LE.

When the Integration Service runs in ASCII mode, no additional code page conversions are performed.

Step 8. Perform RDBMS Code Page Conversions

The relational database client process might be configured to perform code page conversions on character data.

For example, the following databases can perform conversions:

- Oracle by using the NLS_LANG environment variable
- DB2 for Linux, UNIX, and Windows by using the DB2CODEPAGE environment variable
- DB2 for z/OS by using the DB2CODEPAGE statement in the DBMOVER configuration file

Relational Access Methods That Describe Columns

Describing Columns in DB2 for Linux, UNIX, and Windows

To determine the PowerExchange code page number (CPN), PowerExchange performs the following actions:

1. Determines the effective CCSID from the DB2CODEPAGE environment variable.
2. Forms an alias by concatenating "CP" and the CCSID.
3. Looks up the CPN from the alias.

The same code page is used for all CHAR and VARCHAR columns.

In the following situations, it is necessary to set DB2CODEPAGE=1208 so that data is exchanged between PowerExchange and DB2 in UTF-8:

- Performing DB2 for Linux, UNIX, and Windows CDC
- Processing Asian multibyte data on a machine that is localized for the U.S. or Western Europe

Informatica recommends that DB2 for Linux, UNIX, and Windows data is processed by using an Integration Service that runs in ASCII mode and uses UTF-8.

Describing Columns in Microsoft SQL Server

In a bulk data movement session, the code page for character columns is derived from the collation for the database.

In a CDC session, the code page for character columns is always UTF_16LE (Unicode Transformation Format - 16-bit Little Endian).

If SQL Server data is processed under an Integration Service that runs in Unicode mode, Informatica recommends that you use a connection code page of UTF-16LE. With this code page, only half the conversions are performed for bulk processing and almost none for CDC.

Describing Columns in Oracle

Code Page for Exchanging Data

The code page that Oracle uses to exchange data between the database and a calling application, such as a native driver, Data Direct ODBC, SQL Developer, or Toad, is derived from the third part of the NLS_LANG environment variable.

For example, UTF8 is derived from the following NLS_LANG value:

```
NLS_LANG=American_America.UTF8
```

Server Code Page for Physical Data Storage

The character sets in which the data is physically stored are defined when a database is created and used for all tables. The following types of character sets are defined:

- Single-byte character set, which is used for CHAR and VARCHAR2 columns
- Multibyte character set, which is used for NCHAR and NVARCHAR columns

You can determine the character sets by querying the NLS_DATABASE_PARAMETERS table. Use queries such as the following ones:

```
select value from NLS_DATABASE_PARAMETERS where parameter = 'NLS_CHARACTERSET';

select value from NLS_DATABASE_PARAMETERS where parameter = 'NLS_NCHAR_CHARACTERSET';
```

The default values are:

NLS_CHARACTERSET = US7ASCII

NLS_NCHAR_CHARACTERSET = UTF8

Oracle does not perform code page conversions if the character set of the NLS_LANG environment variable is the same as the character set in which the data is physically stored. This feature allows you to physically store data that is not in the expected code page. For example, multibyte data can be stored in a system that expects US_ASCII.

Bulk Data Movement Mode

PowerExcange bulk processing uses a PowerExchange access method that calls Informatica native driver libraries, such as libpmora8.so.

PowerExchange usually determines the code page of columns from the NLS_LANG environment variable. You can force the use of a code page by defining the ORACLECODEPAGE statement in the DBMOVER configuration file, for example:

```
ORACLECODEPAGE=(KO102DTL,MS949,MS949)
```

CDC Mode

Both PowerExchange Oracle CDC implementations, Express CDC for Oracle and Oracle CDC with Logminer, return all column data in UTF-8.

Informatica recommends that you process Oracle data by running the Integration Service in ASCII mode, using UTF-8.

Describing Columns in DB2 for z/OS

For each column that PowerExchange describes in a bulk data movement session, PowerExchange performs the following actions:

- Obtains the CCSID of the column from the catalog tables.
- Derives the effective CCSID after consideration of the DB2CODEPAGE statement.
- Forms an alias by concatenating "CP" and the CCSID.
- Performs a lookup on the alias to return the CPN. ICUCHECK Report 5, "Power Exchange Code Page Names and Aliases," shows the PowerExchange CPNs and associated aliases.

For PowerExchange bulk data movement sessions, the DB2CODEPAGE statement in the DBMOVER configuration file on the z/OS machine controls code page processing.

The statement has the following syntax:

```
DB2CODEPAGE=(db2_subsystem
            [,DB2TRANS={P|N|R}]
            [,MIXED={N|Y}]
            [,EBCDIC_CCSID=({sbcs_ccsid|037}
                           ,{graphic_ccsid|037}
```

```
                ,{mixed_ccsid|037})]
[,ASCII_CCSID=({sbcs_ccsid|850}
               ,{graphic_ccsid|65534}
               ,{mixed_ccsid|65534})]
[,UNICODE_CCSID=({sbcs_ccsid|367}
                 ,{graphic_ccsid|1200}
                 ,{mixed_ccsid|1208})]
[,PLAN_CCSID=({sbcs_ccsid|037}
              ,{graphic_ccsid|037}
              ,{mixed_ccsid|037})]
[,REMAPn=(current_data_ccsid),(remapped_data_ccsid)  )
```

DB2TRANS can have the following values:

DB2TRANS=P

When DB2TRANS=P (the default), DB2 translates the code pages in which column data is stored into the code pages defined in the DB2 plan that was bound for PowerExchange. You must also specify the EBCDIC_CCSID parameter. You can optionally specify the PLAN_CCSID parameter to tell DB2 to convert from one EBCDIC code page to another. If you specify both, the PLAN_CCSID parameter takes precedence. The PLAN_CCSID parameter is useful if PMICU does not support the EBCDIC code page in which the data is stored.

If you have ASCII and Unicode data, you can also specify the ASCII_CCSID and UNICODE_CCSID parameters to map to the EBCDIC code pages.

DB2TRANS=N

When DB2TRANS=N, DB2 does not translate the code pages of the column data to equivalent EBCDIC code pages. PowerExchange uses the native code page in which the data is stored. You do not need to define the EBCDIC_CCSID, ASCII_CCSID, UNICODE_CCSID, or PLAN_CCSID parameters.

When the data is stored in ASCII and you want to pass it through to an application on Linux, UNIX, or Windows, this option can be useful for the following reasons:

- Fewer conversions result in better performance.
- ASCII multibyte data column data might expand when converted to EBCDIC with shift out (X'0E') and shift in (x'0F') characters, making it impossible to process using the plan code page.

DB2TRANS=R

DB2TRANS=R works in a manner similar to DB2TRANS=N. For most columns, DB2 gives and receives data in the CCSID in which the data is actually stored without any translation. However, DB2 translates certain user-specified data code pages to other code pages, as defined in one or more REMAP*n* parameters. In each REMAP*n* parameter, the first positional parameter identifies a data code page to remap, and the second positional parameter identifies the code page to use. Use a code page other than the code page in which the PowerExchange DB2 plan is bound.

The DBTRANS=R option with the REMAP*n* parameter can be useful in the following situations:

- An ASCII double-byte code page is needed for Asian multibyte data.
- Column data is present in a CCSID that PMICU does not support. Without the DB2TRANS=R option, you would have to define the the ICUALIAS statement as a workaround.

Example:

```
DB2CODEPAGE=(D91G,DB2TRANS=R,REMAP1=(301,1200))
```

This example causes DB2 to use the native CCSID for all columns except when the CCSID is 301, in which case DB2 translates CCSID 301 to UTF-16BE (double-byte Unicode).

Nonrelational Access Methods

NRDB Description of Character Columns from Record Fields

For column data that is derived from fields in the record, the internal code page number of CHAR and VARCHAR fields is determined from the following sources, in order of priority:

1. The code page that you select on the **Code Page** tab of the **Field Properties** dialog box in the PowerExchange Navigator.
2. The code page that you select from the **Code Page** list on the **Access Method** tab of the **Data Map Properties** dialog box in the PowerExchange Navigator.
3. The data code page that is derived from the CODEPAGE statement for the PowerExchange Listener in the DBMOVER configuration file.

When a CDC extraction map is created, field-level code pages are defined according to the column attributes of the underlying database.

NRDB Description of Character Columns from User-Defined Fields

For column data derived from user-defined fields, it is not possible to define a code page at the field level. The internal code page number of CHAR and VARCHAR fields is determined from the following sources, in order of priority:

1. The code page that you select from the **Code Page** list on the **Access Method** tab of the **Data Map Properties** dialog box
2. The data code page that is derived from the CODEPAGE statement for the PowerExchange Listener

Some user-defined fields produce results from literals from programs that are vended by third parties. On Linux, UNIX, and Windows, the character data is restricted to 7-bit ASCII values.

Special NRDB Situations

Review the following special situations regarding code pages for NRDB data sources.

Processing z/OS Files Copied by Binary FTP to Linux, UNIX, or Windows

If record ID conditions are used on ZONED or UZONED fields, the comparisons are performed with both the field data and the data map comparison string in ISO-8859.

If record ID conditions are performed on CHAR or VARCHAR fields, the comparisons are performed with both the field data and the data map comparison string in UTF-8.

Processing Danish AE Characters in NRDB SEQ Files on z/OS

NRDB processing usually works best with a SQL code page of IBM-1047 or a code page that is identical to IBM-1047 for the characters that are actually used.

A problem situation occurs when one of the following conditions exists:

- A SQL code page is used that is significantly different from IBM-1047. For example, DB2 for z/OS requires a SQL code page such as CP1142 for Danish data.
- Table and column names contain characters for which the hexadecimal values differ between IBM-1047 and the SQL code page, such as the Danish AE character.

To handle this situation, PowerExchange performs the following actions when processing NRDB data:

- At the beginning of Describe processing, the column names derived from parsing the SQL are converted to the metadata code page.
- From that point on, all internal processing is performed in the metadata code page, especially when matching to data map objects.
- If column names are used in error messages and the control code page differs from the metadata code page, the metadata code page is converted to the control code page.

Character Column Contains Packed or Integer Data

Problems can occur when a packed column contains packed or integer data. This situation can also occur in DB2 but is more common with NRDB sources.

The following problems can result:

- Termination of a character string when a binary zero character is encountered when processing a PowerCenter workflow. This situation can occur if you do not specify `PreserveLowValues=Yes` in the **Custom Properties** field of the **Config Object** tab in the Powercenter Workflow Manager.
- Termination of a character string when a binary zero character is encounterd in PowerExchange processing. This situation can occur if you do not specify LOWVALUES=Y in the DBMOVER configuration file.
- Substitution characters if a one-to-one mapping does not exist between the code page of the caller and the code page of the access method, such as when the caller uses the default MS1252 (Latin 1) and the access method uses IBM-037.

z/OS Considerations

DB2 for z/OS ECCR

The DB2 for z/OS ECCR has no intelligence of code pages and assumes that all names are in the same code page that is used to exchange data with DB2, that is, the code page of the DB2 plan.

A problem occurs if table and column names in DB2 use characters that have different values in IBM-1047 and the DB2 code page. PowerExchange metadata in the CCT file is stored in the metadata code page IBM-1047 on z/OS. If names are not converted to the DB2 code page, processing fails when the table and column names in registrations cannot be found in the DB2 catalog tables.

The required conversion is located in the PowerExchange Agent task. The Agent task determines the code page that the DB2 ECCR uses from the EBCDIC values in the DB2CODEPAGE statement for the subsystem in the DBMOVER configuration file. In this statement, you must specify the EBCDIC_CCSID subparameter and you must *not* specify a DB2TRANS value of N or R.

Single-Byte Metadata Limitation

PowerExchange defines metadata in the single-byte code page IBM-1047 on z/OS.

Defining metadata in the IBM-1047 code page prevents the following events from occurring:

- Processing of multibyte table and column names in DB2
- Processing of single-byte table and column names if the characters are not present in code page IBM-1047

For a list of characters that are supported in z/OS metadata and are outside of the 7-bit ASCII range, see "Appendix A: EBCDIC Metadata Characters outside US_ASCII" on page 27.

Only single-byte code pages are allowed in the ctrl_cp and SQL_cp parameters of the CODEPAGE statement in the DBMOVER configuration file.

On Linux, UNIX and Windows, PowerExchange stores data maps in the UTF-8 code page, which can handle all table and column names. On z/OS, no single code page exists that can handle all characters.

PMICU Usage on z/OS

PowerExchange is designed to use PMICU as little as possible on z/OS for the following reasons:

- Each subtask using PMICU uses a 9-MB region in addition to what PowerExchange requires.
- Code page processing consumes a lot of CPU, which is expensive on z/OS.

PMICU is used on z/OS in the following areas:

- Conversion of user IDs and passwords from UTF-8 after decryption from AES. PMICU is used if any characters are outside of the 7-bit ASCII range.
- Conversion of SQL characters between the SQL code page and the metadata code page (IBM-1047).
- Conversion of character fields with record ID conditions if the field is not in IBM-1047.
- Processing of DB2 registrations if the DB2CODEPAGE statement indicates that DB2 uses a code page other than IBM-1047 or IBM-037. This affects the PowerExchange Agent started task and PowerExchange utilities, such as DTLUCBRG.

PMICU

PMICU Background

PMICU is an Informatica software system that PowerCenter uses for code page conversions. It is based on International Components for Unicode (ICU).

ICU is open source software. For more information about ICU, see http://site.icu-project.org/.

PowerCenter 8.5.1 and later releases use PMICU for code page conversions.

PMICU is based on the ICU 3.2.1 system plus the following features:

- Additional customized code pages included in the data library
- CPU optimizations
- Renaming of libraries to include an additional PM prefix, which reduces the chance of a library name collision with pre-existing ICU libraries on the path

PowerExchange releases 5.2.1, 8.1.1, and 8.5.1 used ICU V2.6.1. PowerExchange releases 8.6.0 and later use PMICU at the same release level as that used by PowerCenter.

In PowerExchange contexts, it is possible to use PowerCenter short names or long descriptions as well as ICU converter names and aliases.

Substitution Characters

An incorrect code page configuration can result in substitution characters.

When the ucnv_fromUChars() conversion routine cannot convert characters in character data during the PowerExchange Open call (for example, the user ID, database name, or table name), processing is aborted and error message PWX-1291 is issued. This problem occurs because the default behavior on character substitutions has been overridden.

When ucnv_fromUChars() cannot convert characters in column data, it quietly replaces them with the substitution character, which is sometimes displayed as a question mark.

The following table shows example substitution values:

Code Page	Hexadecimal	Unicode	Display	Description
ASCII ANSI code pages	X'1A'	<U001A>		Up arrow
ASCII OEM code pages	X'7F'	<U001A>		Up arrow
EBCDIC code pages	X'3F	<U001A>		Up arrow
UTF8	X'EFBFBD'	<U00FFFD>		Question mark enclosed in by kite shape
UTF-16BE	X'FFFD'	<U00FFFD>		Question mark enclosed in a kite shape
UTF-16LE	X'FDFF'	<U00FFFD>		Question mark enclosed in a kite shape
Double-byte code pages	X'FEFE'			
Double-byte code pages	X'FCFC'			
Double-byte code pages	X'F4FE'			

Substitution values for each code page are specified in the UCM file when the code page is created. They can be reported by the ICU API ucnv_getSubstChars() routine and the utility ICUINFO.EXE in "Report 1: Converter Information" column "Sub char."

The following considerations apply to substitution characters:

- Substitution characters indicate data loss. Although tolerating data loss is the default behavior, data loss is rarely acceptable.
- The way substitution characters are displayed can be misleading and depends on contexts and fonts.
- Substitutions often occur when processing IBM-037 EBCDIC data using an Integration Service that runs in Unicode mode with the default "MS Windows Latin 1 (ANSI), superset of Latin1" connection code page. Using ISO-8859 fixes this problem.
- Other combinations of EBCDIC and ASCII code pages exist where substitutions can occur. It is time-consuming to identify whether particular characters are present in both code pages.
- A smaller chance of substitution problems exists if fewer code page conversion steps are involved, such as when you run workflows in ASCII mode or use UTF-16BE/LE in Unicode mode.

Supplemental Characters

The Unicode Basic Multilingual Plane (BMP) includes 55,237 characters. When the Integration Service is running in Unicode mode, each character is formed from a single NUM16 integer.

Beyond the BMP, characters in supplemental planes are formed from two NUM16 Integers. Informatica does not officially support these supplemental characters, because they can cause problems in determining the column size.

Supplemental characters occupy 4 bytes in UTF-8, such as hexadecimal 'F0A08080' for character <U0010000> LINEAR B SYLLABLE B008 A.

Approximately 96,689 characters are available in the code pages that support the entire Unicode range, such as UTF-16BE, UTF-16LE, and UTF-8.

A number of Asian code pages use supplemental characters, including gb18030 and IBM-16684.

Customized ICU Code Pages

Customized code pages are useful in the following situations:

- You are using a code page that PMICU does not support.
- You are trying to carry EBCDIC text formatting into an ASCII environment.
- You have problem data that needs to be cleaned.
- IBM EBCDIC code pages and Linux, UNIX, or Windows ASCII code pages use different characters for the Asian full- and half-width dash and tilde.

When reading, it is possible to map several characters in the source code page to the same Unicode target. In addition to the primary round-trip mapping, you can create additional one-way mappings by using type 3 fallback mappings.

When writing, it is possible to map several Unicode characters to the same target code page hexadecimal value. In addition to the primary round-trip mapping, you can create additional one-way mappings by using type 1 fallback mappings.

In many cases, ICU off-the-shelf code pages can handle problem situations, and the access methods can be configured to describe column data differently from what the database expects.

It is *not* possible to use customized ICU code pages to fix corrupted data, such as the data that results when double-byte and mixed-byte EBCDIC data has been stored in the same column.

Non-ICU Code Pages

In PowerExchange releases prior to 5.2.1, only a limited number of single-byte code pages were supported, as shown in the following table. Code page conversion was performed by building a 256-byte translate table based on the source and target code pages in relation to a base of ISO-8859.

CPN	Name	Type
001	ISO-8859	ASCII
002	IBM-1047	EBCDIC
003	IBM-037	EBCDIC
004	IBM-273	EBCDIC
005	IBM-500	EBCDIC
006	IBM-284	EBCDIC
007	IBM-297	EBCDIC
008	IBM-280	EBCDIC
009	IBM-285	EBCDIC
010	IBM-277	EBCDIC
011	IBM-278	EBCDIC
012	PC-856	ASCII
013	IBM-424	EBCDIC

CPN	Name	Type
014	IBM-870	EBCDIC
015	MS-1250	ASCII
016	HP-ROM8	ASCII

These old code pages are sometimes referred to as static or simple code pages. Each has an equivalent ICU code page.

Static code page conversion is performed if neither code page is an ICU code page. If either the source or the target code page is an ICU code page, conversion is performed by using the ICU APIs by means of intermediate Unicode.

Since PowerExchange 9.0.1, conversion between two single-byte ICU code pages has been optimized to use a 256-byte translate table built from the ICU APIs.

Code Page Usage by Country, Language, and Type

Code Page Usage Reports

The following resources provide information about code page usage by country, language, and type:

- The "Globalization" chapter of the *PowerExchange Reference Manual*
- Report 2 from the ICUCHECK utility

EBCDIC Code Pages that Support the Euro Sign

A set of code pages have replaced the currency sign with the euro sign. Generally, you should use these code pages instead of the equivalent code pages without the euro sign. The following code pages include the euro sign:

223 IBM-1140 EBCDIC US (with euro update)

224 IBM-1141 EBCDIC Germany, Austria (with euro update)

225 IBM-1142 EBCDIC Denmark, Norway (with euro update)

226 IBM-1143 EBCDIC Finland, Sweden (with euro update)

227 IBM-1144 EBCDIC Italy (with euro update)

228 IBM-1145 EBCDIC Spain, Latin America (with euro update)

229 IBM-1146 EBCDIC UK, Ireland (with euro update)

230 IBM-1147 EBCDIC French (with euro update)

231 IBM-1148 EBCDIC International Latin1 (with euro update)

232 IBM-1149 EBCDIC Iceland (with euro update)

233 IBM-1153 EBCDIC Latin2 (with euro update)

234 IBM-1154 EBCDIC Cyrillic Multilingual (with euro update)

235 IBM-1155 EBCDIC Turkey (with euro update)

236 IBM-1156 EBCDIC Baltic Multilingual (with euro update)

237 IBM-1157 EBCDIC Estonia (with euro update)

238 IBM-1158 EBCDIC Cyrillic Ukraine (with euro update)

239 IBM-1159 IBM EBCDIC Taiwan, Traditional Chinese

240 IBM-1160 EBCDIC Thai (with euro update)

241 IBM-1164 EBCDIC Vietnamese (with euro update)

Common Single-Byte Code Pages

The following table lists the most common single-byte code pages:

CPN	Converter Name	Short Name	Long Name
ASCII	ibm-5348_P100-1997	MS1252	MS Windows Latin 1 (ANSI), superset of Latin1
ASCII	ISO-8859-1	Latin1	ISO 8859-1 Western European
ASCII	UTF8	UTF8	UTF-8 encoding of Unicode
ASCII	US-ASCII	US-ASCII	7-bit ASCII
EBCDIC	ibm-37_P100-1995	IBM037	IBM EBCDIC US English IBM037
EBCDIC	ibm-1047_P100-1995	IBM1047	IBM EBCDIC US English IBM1047

Turkish EBCDIC Code Pages

The double-quotation mark character and logical NOT sign can be problematic in Turkish EBCDIC code pages.

The double-quotation mark character is not at the standard hexadecimal location for EBCDIC Turkish code pages CP1026 and CP1155. This situation causes problems when the PowerExchange input parser parses SQL that should contain the double-quotation mark character with the hexadecimal value that is in the code page in which the PowerExchange DTLBASE module is compiled (IBM-1047 on z/OS).

PowerExchange works around the problem in the following ways:

- When SQL is sent from the calling application, PowerExchange replaces the problem character X'FC' with its standard EBCDIC equivalent X'7F'.
- In z/OS programs, PowerExchange treats both X'7F' and X'FC' as meaning <U0022> " QUOTATION MARK.

This method makes it impossible to use character <U00DC> Ü LATIN CAPITAL LETTER U WITH DIAERESIS in Turkish DB2 names.

Another problem affects the logical NOT sign in JCL. After an FTP transfer from Windows to a z/OS system that runs CP1026 or CP1155, this character becomes a value that the JES reader does not recognize. PowerExchange works around this problem by constructing JCL that uses the explicit NOT literal rather than the one-byte <U00AC> ¬ NOT SIGN.

Japanese EBCDIC Code Pages

The following mixed-byte code pages are in widespread use on Japanese z/OS machines:

- CP930, which has nonstandard lowercase a to z characters
- CP939, which has standard lowercase a to z characters

When possible, use CP939 instead of CP930. However, use of CP939 is not possible in some situations, such as when DB2 on z/OS uses CCSID 930.

CP930 has lowercase letters a to z in nonstandard locations. In standard EBCDIC, which DB2 uses, these characters lie in the ranges X'81 - X'89', X'91 - X'99', and X'A2 - X'A9'. However, Japanese CP930 stores Katakana characters at these locations. To work around the problem, PowerExchange converts SQL to uppercase before performing a code page conversion if the z/OS Listener uses CP930. This conversion means that SQL literals are passed to DB2 in uppercase, for example. as SELECT, FROM, WHERE, and INSERT.

The workaround enables SQL to be processed correctly most of the time. However, when CP930 is used, it is not possible to process tables and columns with case-sensitive lowercase names.

Arabic and Hebrew EBCDIC

Arabic script presents the following problems to people who are unfamiliar with it:

- On Windows, the letters in cursive script are joined together so that it is difficult or impossible to know where one character ends and the next one begins.
- The text is rendered from right to left, whereas digits are rendered left to right as in Western contexts.

On z/OS EBCDIC systems, code page CP420 and its associated fonts present Arabic characters using a standalone format known as "shaped," in which no lines join the preceding and following characters. In effect, z/OS systems use primitive 1970s technology, and Arabic readers have difficulty reading reports from mainframe printers.

On Linux, UNIX, and Windows systems, Arabic is presented in a cursive style in which the fonts join characters in a way that is similar to how handwritten characters are joined. This method is regarded as more technically correct, because Arabic characters should appear joined.

To convert Arabic from EBCDIC to ASCII, PowerExchange must perform the following types of conversions:

- Replace characters. That is, different Unicode values must be used. PowerExchange supports this conversion by using the following DBMOVER statement:

  ```
  ICUCNVPROPERTY=(198,UNSHAPE_ARABIC,ON)
  ```

- Insert space characters in the correct places so that no cursive connecting strokes occur between words. PowerExchange supports this conversion by using the following DBMOVER statement:

  ```
  ICUCNVPROPERTY=(198,IBM420_END_OF_WORD_SPACES,ON)
  ```

- Reverse digits. For example, "123" is changed to "321". PowerExchange supports this conversion by using the following DBMOVER statement:

  ```
  ICUCNVPROPERTY=(198,REVERSE_EBCDIC_DIGITS,ON)
  ```

In summary, the following DBMOVER statements are required to convert from EBCDIC code page IBM-420 to a Linux, UNIX, and Windows code page such as MS1256:

```
ICUCNVPROPERTY=(198,REVERSE_EBCDIC_DIGITS,ON)
ICUCNVPROPERTY=(198,IBM420_END_OF_WORD_SPACES,ON)
ICUCNVPROPERTY=(198,UNSHAPE_ARABIC,ON)
```

ICUCNVPROPERTY is applied against the internal code page number 198, which uses converter ibm-420_X120-1999. ICUCNVPROPERTY might also be applied against other EBCDIC Arabic code pages such as CPN 253 and CPN 267, because IBM-16804 is related to IBM-420 (16804 = 4096 *4 + 420).

The REVERSE_EBCDIC_DIGITS property can also be used for EBCDIC Hebrew code pages such as CPN 199, which uses converter ibm-424_P100-1995.

Issues That Have Workarounds

Non-conversion of Control Characters

When the EXT_CP_SUPPT statement in the DBMOVER configuration file has a value of N, characters less than an EBCDIC space character in PowerExchange static code pages are not translated. This issue arose in PowerExchange releases earlier than 9.6.0, because the EXT_CP_SUPPT statement had a default value of N in these releases. In PowerExchange 9.6.0 and later, the default value is Y.

When the EXT_CP_SUPPT statement has a value of N, certain problematic hexadecimal values remain at the same hexadecimal values after conversion. As a result, the following problems can occur:

- You cannot not write back to get the same values that you started with. That is, you cannot perform a round-trip mapping.
- The hexadecimal values differ from those that result from proper ICU conversions.

For example, suppose you are converting from the source code page IBM-037 CPN 3 to the target code page ISO-8859 CPN 1.

Because the following character in IBM-037 is not converted, it remains at X'25'.

```
X'25' <U000A> LINE FEED (LF)
```

In ISO-8859, X'25 represents the following character:

```
X'25 <U0025> % PERCENT SIGN
```

This conversion problem would not have occurred if the ICU APIs were used.

Although the default was changed in PowerExchange 9.6.0, in certain cases, you might want to use nonstandard code page mappings. In this case, use one of the following workarounds:

- Use customized ICU code pages.
- Set the EXT_CP_SUPPT statement to N to disable the conversion of control characters. However, this approach does not work if the Integration Service runs in ASCII mode or if an ICU connection code page is used. For this reason, Informatica recommends using customized ICU code pages instead of this workaround.

Truncation of Strings at the First Binary Zero Character

By default, both PowerCenter and PowerExchange truncate data at the first binary zero character. This truncation causes the following problems:

- Loss of characters beyond the truncation point
- Processing cost of executing strlen() or the Unicode equivalent

The problem has been reported by customers who were processing packed or integer data in CHAR columns and required the data to be passed through the mappings without anything getting lost when the data was written back to a key field in a KSDS file or DB2 table.

To avoid this problem, perform one of the following actions or sets of actions:

- In the NRDB data map, remap the problem column as a column with the BIN or VARBIN datatype. This solution is not possible with DB2 tables.

- Perform both of the following steps:
  - Specify `PreserveLowValues=Yes` in the **Custom Properties** field on the **Config Object** tab in the Powercenter Workflow Manager.
  - Specify LOWVALUES=Y in the DBMOVER configuration file on the Integration Service machine.

Unable to Start ASCII Mode Integration Service in Certain Code Pages

In some situations, it makes sense to run a PowerCenter ASCII mode Integration Service in a particular code page to get minimal conversions and avoid substitution characters.

Consider the following examples:

- Using an ASCII mode Integration Service in UTF-8 to process Oracle or DBD for Linux, UNIX, and Windows CDC data.
- Use an ASCII mode Integration Service in UTF-8 to process Oracle bulk data with NLS_LANG set to UTF-8.
- Use an ASCII mode Integration Service in ISO-8859 to process z/OS data in IBM-037.

The PowerCenter Integration Service includes a validation of the code page that is selected for the Integration Service against the OS locale of the machine together with the code page that the Java Tomcat services use. In problem situations, the Integration Service does not start, and an obscure message is logged on the domain, such as the following message:

```
Unable to start the integration service on any node.
```

To work around the problem, perform the following actions:

1. Shut down the Tomcat services. Use the infaservice shutdown command.
2. Set the environment variable INFA_CODEPAGENAME to the required code page. Use the SET INFA_CODEPAGENAME=UTF-8 command.
3. Restart the Tomcat services. Use the infaservice startup command.

You can also work around the limitation by running workflows in Unicode mode. However, this approach causes extra conversion steps and performance and substitution side effects.

Limitations

Unable to Truncate Multibyte Column Data

PowerExchange is able to truncate column data when the output code page is single byte because the character boundaries are easy to determine. With multibyte code pages, the character boundaries are more difficult to determine. The current implementation of the PowerExchange API passes an output buffer that is sized at the maximum number of characters allowed in the output column. If the PMICU routines determine that not enough space is available to hold the converted data, error U_BUFFER_OVERFLOW is flagged and processing aborts.

Multibyte Precision Not Known After Conversion

The maximum size of a column is hard to determine in situations where column data is converted to a target multibyte code page. For example, when the Integration Service runs in Unicode, code page conversion to UTF-8 mode might require three times the original number of bytes.

The following factors worsen the problem:

- PowerExchange does not support truncation. Processing aborts when a buffer overflow condition is met.

- The PowerExchange API appends trailing spaces to CHAR columns when reading data.

To work around these limitations, PowerExchange performs the following actions:

- To avoid truncation, when importing source metadata, PowerExchange sets the size of columns in source objects to handle the greatest possible expansion. The formula takes into account the minimum character size in the remote database as described by the access method and the maximum character size in the mapping. For example, PowerExchange multiplies the precision by three when converting to UTF-8.
- To avoid appending unwanted spaces in CHAR columns, in certain cases at run time, PowerExchange describes CHAR column data as VARCHAR client fields.
- To avoid truncation, at run time, PowerExchange logs error message PWX-07096 if data might overflow the PWXPC buffers that the caller has bound.

Example PWX-07096 message:

```
07096 column number (name) caller buffer length buffer_length less than the minimum minimum for
multibyte code page pwx_code_page_number (name).
```

The PWX-07096 message provides the following information:

- Column number and name
- Buffer size provided by the caller to receive the data
- Larger buffer size that DTLSCLI requires
- Code page to receive the data, which is the Integration Service code page in ASCII mode or the connection code page in Unicode mode

PowerExchange issues the PWX-07096 message only when reading. PowerExchange issues the message either because of an explicit bind during the initialization or because of a deferred bind at the time of the first read. The error indicates that data might overflow into subsequent rows if the column data is expanded to the maximum number of bytes. For example, 3 bytes are used in UTF-8 for Japanese characters and for the substitution character.

The formula for determining the maximum number of bytes derives an expansion factor from the minimum number of bytes in a character in the source code page and the maximum number of bytes in a character in the target code page. Thus, an expansion of three times is possible if converting from ISO-8859 to UTF-8.

Unable to Process Different Code Pages Inside a Single Column

You might have files and tables that contain data in different code pages. This situation can be problematic in a PowerCenter workflow, where all column data must be in the same code page. In some cases, PowerExchange can process the data by using record ID conditions based on code page identifiers in another part of the same record. But in other cases, such processing is not possible.

Frequently Asked Questions

Where are code page conversions performed?

Code page conversions are performed in the client process and not in the Listener.

Because PowerCenter and the Informatica client processes run on Linux, UNIX, or Windows, they do not incur the CPU usage charges for code page conversions, which can occur on z/OS.

Example 1. A PowerCenter workflow reads column data from a DB2 for z/OS source when the Integration Service runs on a Linux machine

Code page conversions are performed on the Linux machine where the Integration Service runs.

On z/OS, the DB2 access method describes the code page of the column data, but no conversions are performed there.

Example 2. A PowerCenter workflow writes column data to a DB2 for z/OS target when the Integration Service runs on a Linux machine

Code page conversions are performed on the Linux machine where the Integration Service runs.

This example is similar to example 1.

Example 3. Row test of DB2 for z/OS data in the PowerExchange Navigator

Code page conversions are performed on the Windows machine where the PowerExchange Navigator runs.

Example 4. PowerExchange DTLURDMO utility processes DB2 metadata through a Listener on z/OS

Code page conversions are performed in the DTLURDMO process and not in the Listener.

If DTLURDMO runs as a batch job on z/OS and code page conversions are required (for example, because DB2 column names are not in CP1047), the CPU cost is clocked against the DTLURDMO job.

If DTLURDMO runs on Linux, UNIX, or Windows, the code page conversions are performed on the Linux, UNIX, or Windows machine.

What is the recommended data movement mode for the Integration Service?

The recommended data movement mode for the Integration Service depends on the circumstances.

If you are processing single-byte data and no issues exist with substitution characters when converting between EBCDIC and ASCII characters, ASCII mode is recommended. ASCII mode has the following advantages:

- Fewer conversions are performed.
- Performance is slightly faster.
- Because column data occupies fewer bytes, DTM buffers are used more efficiently.

Unicode mode is preferred under any of the following conditions:

- Multibyte data is being processed.
- Issues with column precisions exist.
- Data from different sources and code pages is being consolidated.

Profiling has shown that ASCII mode typically consumes between 10% and 20% less CPU and elapsed time.

When possible, use Unicode mode with connection code pages of UTF-16LE or UTF-16BE. Profiling shows that this configuration performs about 10% better than using UTF-8 as the connection code page. This configuration also avoids the problem of choosing the wrong connection code page, which can result in substitution characters.

Can PowerExchange read multibyte file names?

On Windows, the PowerExchange Navigator can open and read files with multibyte names. The Navigator forces the control code page to UTF-8 and uses the Windows API _wfopen().

A PowerExchange Listener on Windows can read a multibyte file name provided that the CODEPAGE statement in the DBMOVER configuration file defines a suitable multibyte control code page.

On Linux and UNIX, the situation is less clear. The ANSI API fopen() might work if the control code page in the CODEPAGE statement agrees with the machine locale in which files are named.

Can the PowerExchange Navigator display text in a language for which PowerCenter is not localized?

By default, the PowerExchange Navigator displays resources according to the localization option that was selected when Windows was installed. Reconfiguring that option is difficult within Windows.

In certain situations, you might want to emulate foreign localization. For example, suppose you use an Asian language and want to report a PowerExchange Navigator problem to Informatica Global Customer Support. Customer Support personnel might prefer to see English text on Navigator screens, but the machine is not configured to run the PowerExchange Navigator in English.

BAT files for running the PowerExchange Navigator with different localized resources are available.

BAT file syntax using language 1033, PWXRES409.DLL, and the English DTLMSG file

```
set DTL_UILanguage=1033
set DTL_DTLMSG_CODEPAGE=none
set DTL_DTLMSG_LANGUAGE=
echo Resources DLL=PWXRES409.DLL
echo Message text in dtlmsg.txt
start "dtlui" dtlui.exe
```

BAT file syntax using language 1041, PWXRES411.DLL, and the Japanese DTLMSG file

```
set DTL_UILanguage=1041
set DTL_DTLMSG_CODEPAGE=shiftJIS
set DTL_DTLMSG_LANGUAGE=
echo Resources DLL=PWXRES411.DLL
echo Message text in dtlmsg_shiftJIS.txt
echo Calling DTLUI.EXE
start "dtlui" dtlui.exe
```

Other countries follow a similar model.

For more information, contact Informatica Global Customer Support.

Can PowerExchange process multibyte Asian data on a U.S. localized machine?

The way that a machine is localized has no effect on the way that column data is processed. The underlying PMICU and PowerExchange conversion APIs work independently of localization. You can run workflows on Linux, UNIX or Windows machines to process data from any part of the world. Although it is technically possible to run client applications on i5/OS or z/OS to process multibyte data, this approach would be costly because of CPU-based charging when processing is not performed on zIIP processors.

When multibyte Asian data is processed on a U.S. localized machine, the following limitations apply to how substitution values in error messages are displayed:

- Listener displays in command sessions on Linux, UNIX, or Windows typically produce a large number of undisplayable characters. Although this problem can be reduced by using the CONSOLE_CODEPAGE statement in the DBMOVER configuration file, the problem cannot be eliminated.
- Problems occur if the detail.log file contains text in a mixture of code pages. You must use the LOG_CODEPAGE statement in the DBMOVER configuration file to make the log consistent.

What are the Unicode code pages to use and to avoid?

Use the following Unicode code pages:

- UTF-8 in contexts where NULL terminated strings are needed, for example, control and SQL code pages
- UTF-16BE or UTF-16LE in contexts where data is being processed as NUM16 integers, for example, the Connection code page for an Integration Service that runs in Unicode mode

Avoid the following Unicode code pages:

- Any code pages with "UTF-32" in the name. It is rare to represent characters as 32-bit integers. Such a representation is probably a mistake.
- Any code pages with "Opposite Endian" in the name. There are few contexts in which it is useful to flip the endianness of NUM16 integers to a style that cannot be processed on the machine.
- UTF-16. This code page causes a byte order mark (BOM) prefix to be added at the front of data in UTF-16BE or UTF-16LE, in a manner similar to how notepad.exe adds one. This method works with a single BOM at the front of a file. It does not work well with column data for the following reasons:
  - Most components reading Unicode data fail because they are not expecting the BOM.
  - The BOM wastes space and causes difficulties in determining the maximum column size.

How many bytes does a wchar_t character contain?

On Windows x86 and x64, a wchar_t type character always contains 2 bytes. Consequently, it is possible to determine the length of a Unicode string in UTF-16LE by using the function wcslen() and to convert between Unicode and ANSI characters by using the function swprintf(,"%S",).

On Linux and UNIX machines, the size of a wchar_t character is not consistent. For example, if the program is compiled and runs on a machine where the locale specifies UTF-8, the size is 4 bytes and UTF-16BE or UTF-16LE characters cannot be processed by the wcslen() or swprintf() functions. Consequently, PowerExchange avoids the use of wchar_t characters when processing column data in Unicode.

Appendix A: EBCDIC Metadata Characters outside US_ASCII

The following characters are allowed in DB2 for z/OS table and column names:

Unicode	Display	Name
<U00A3>	£	POUND SIGN
<U00A5>	¥	YEN SIGN
<U00C0>	À	LATIN CAPITAL LETTER A WITH GRAVE
<U00C1>	Á	LATIN CAPITAL LETTER A WITH ACUTE
<U00C2>	Â	LATIN CAPITAL LETTER A WITH CIRCUMFLEX
<U00C3>	Ã	LATIN CAPITAL LETTER A WITH TILDE
<U00C4>	Ä	LATIN CAPITAL LETTER A WITH DIAERESIS
<U00C5>	Å	LATIN CAPITAL LETTER A WITH RING ABOVE
<U00C6>	Æ	LATIN CAPITAL LETTER AE
<U00C7>	Ç	LATIN CAPITAL LETTER C WITH CEDILLA
<U00C8>	È	LATIN CAPITAL LETTER E WITH GRAVE
<U00C9>	É	LATIN CAPITAL LETTER E WITH ACUTE

Unicode	Display	Name
<U00CA>	Ê	LATIN CAPITAL LETTER E WITH CIRCUMFLEX
<U00CB>	Ë	LATIN CAPITAL LETTER E WITH DIAERESIS
<U00CC>	Ì	LATIN CAPITAL LETTER I WITH GRAVE
<U00CD>	Í	LATIN CAPITAL LETTER I WITH ACUTE
<U00CE>	Î	LATIN CAPITAL LETTER I WITH CIRCUMFLEX
<U00CF>	Ï	LATIN CAPITAL LETTER I WITH DIAERESIS
<U00D0>	Đ	LATIN CAPITAL LETTER ETH
<U00D1>	Ñ	LATIN CAPITAL LETTER N WITH TILDE
<U00D2>	Ò	LATIN CAPITAL LETTER O WITH GRAVE
<U00D3>	Ó	LATIN CAPITAL LETTER O WITH ACUTE
<U00D4>	Ô	LATIN CAPITAL LETTER O WITH CIRCUMFLEX
<U00D5>	Õ	LATIN CAPITAL LETTER O WITH TILDE
<U00D6>	Ö	LATIN CAPITAL LETTER O WITH DIAERESIS
<U00D8>	Ø	LATIN CAPITAL LETTER O WITH STROKE
<U00D9>	Ù	LATIN CAPITAL LETTER U WITH GRAVE
<U00DA>	Ú	LATIN CAPITAL LETTER U WITH ACUTE
<U00DB>	Û	LATIN CAPITAL LETTER U WITH CIRCUMFLEX
<U00DC>	Ü	LATIN CAPITAL LETTER U WITH DIAERESIS
<U00DD>	Ý	LATIN CAPITAL LETTER Y WITH ACUTE
<U00DE>	Þ	LATIN CAPITAL LETTER THORN
<U00DF>	ß	LATIN SMALL LETTER SHARP S
<U00E0>	à	LATIN SMALL LETTER A WITH GRAVE
<U00E1>	á	LATIN SMALL LETTER A WITH ACUTE
<U00E2>	â	LATIN SMALL LETTER A WITH CIRCUMFLEX
<U00E3>	ã	LATIN SMALL LETTER A WITH TILDE
<U00E4>	ä	LATIN SMALL LETTER A WITH DIAERESIS
<U00E5>	å	LATIN SMALL LETTER A WITH RING ABOVE

Unicode	Display	Name
<U00E6>	æ	LATIN SMALL LETTER AE
<U00E7>	ç	LATIN SMALL LETTER C WITH CEDILLA
<U00E8>	è	LATIN SMALL LETTER E WITH GRAVE
<U00E9>	é	LATIN SMALL LETTER E WITH ACUTE
<U00EA>	ê	LATIN SMALL LETTER E WITH CIRCUMFLEX
<U00EB>	ë	LATIN SMALL LETTER E WITH DIAERESIS
<U00EC>	ì	LATIN SMALL LETTER I WITH GRAVE
<U00ED>	í	LATIN SMALL LETTER I WITH ACUTE
<U00EE>	î	LATIN SMALL LETTER I WITH CIRCUMFLEX
<U00EF>	ï	LATIN SMALL LETTER I WITH DIAERESIS
<U00F0>	ð	LATIN SMALL LETTER ETH
<U00F1>	ñ	LATIN SMALL LETTER N WITH TILDE
<U00F2>	ò	LATIN SMALL LETTER O WITH GRAVE
<U00F3>	ó	LATIN SMALL LETTER O WITH ACUTE
<U00F4>	ô	LATIN SMALL LETTER O WITH CIRCUMFLEX
<U00F5>	õ	LATIN SMALL LETTER O WITH TILDE
<U00F6>	ö	LATIN SMALL LETTER O WITH DIAERESIS
<U00F8>	ø	LATIN SMALL LETTER O WITH STROKE
<U00F9>	ù	LATIN SMALL LETTER U WITH GRAVE
<U00FA>	ú	LATIN SMALL LETTER U WITH ACUTE
<U00FB>	û	LATIN SMALL LETTER U WITH CIRCUMFLEX
<U00FC>	ü	LATIN SMALL LETTER U WITH DIAERESIS
<U00FD>	ý	LATIN SMALL LETTER Y WITH ACUTE
<U00FE>	þ	LATIN SMALL LETTER THORN
<U00FF>	ÿ	LATIN SMALL LETTER Y WITH DIAERESIS

Author

Ross Ferrand